The Journal of Open Source Software

QuantizedSystemSolver: A discontinuous ODE system solver for Julia.

Elmongi Elbellili [1,2], **Daan Huybrechs** [2], **and Ben Lauwens** [1]

1 Royal Military Academy, Brussels, Belgium **2** Kuleuven, Leuven, Belgium

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Patrick Diehl

Reviewers:

- @joshuaeh
- @IamBOOO

Submitted: 28 August 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

The growing intricacy of contemporary engineering systems, typically reduced to stiff differential equations with events, poses a difficulty in digitally simulating them using traditional numerical integration techniques. The Quantized State System (QSS) and the Linearly Implicit Quantized State System (LIQSS) are different methods for tackling such problems. It is an approach that builds the solution by updating the system variables independently as opposed to classic integration methods that update all the system variables every step. (Pietro et al., 2019).

Statement of need

Traditional solvers are challenged by frequent discontinuities where the state of the system abruptly changes at specific points or intervals. They struggle to accurately capture the dynamics around discontinuities. They either undergo expensive iterations to pinpoint exact discontinuity instances or resort to interpolating their locations, resulting in unreliable outcomes. Written in the easy-to-learn Julia language Julia programming language (Bezanson et al., 2017), and taking advantage of its features such as multiple dispatch and metaprogramming, the QuantizedSystemSolver.jl is a solver that aims to efficiently solve a set of Ordinary differential Equations with a set of events via implementing the QSS and LIQSS methods. It is the first such tool to be published in the Julia ecosystem.

Package description

While the package is optimized to be fast, extensibility is not compromised. It is divided into 3 entities that can be extended separately: Problem, Algorithm, and Solution. The rest of the code is to create these entities and glue them together as shown in the Figure. The API was designed to provide an easier way to handle events than the approach provided by the classic integration solvers. Inside an NLodeProblem function, the user may introduce any parameters, variables, equations, and events:

```
odeprob = NLodeProblem(quote
          name
          parameters
          discrete and continous variables
          helper expressions
          differential equations
          if-statments for events)
```

The output of this function is an object of type Problem, and it is passed to the solve function along any other configuration arguments such as the algorithm type, the time span and the

tolerance. The solve function dispatches on the given algorithm and start the numerical integration.

```
tspan = (0.0, 0.001)
sol= solve(odeprob,nmliqss2(),tspan,abstol=1e-4,reltol=1e-3)
```

A the end, a solution object is produced that can be queried and plotted.

```
sol(0.0005,idxs=2)
sol.stats
plot(sol)
```

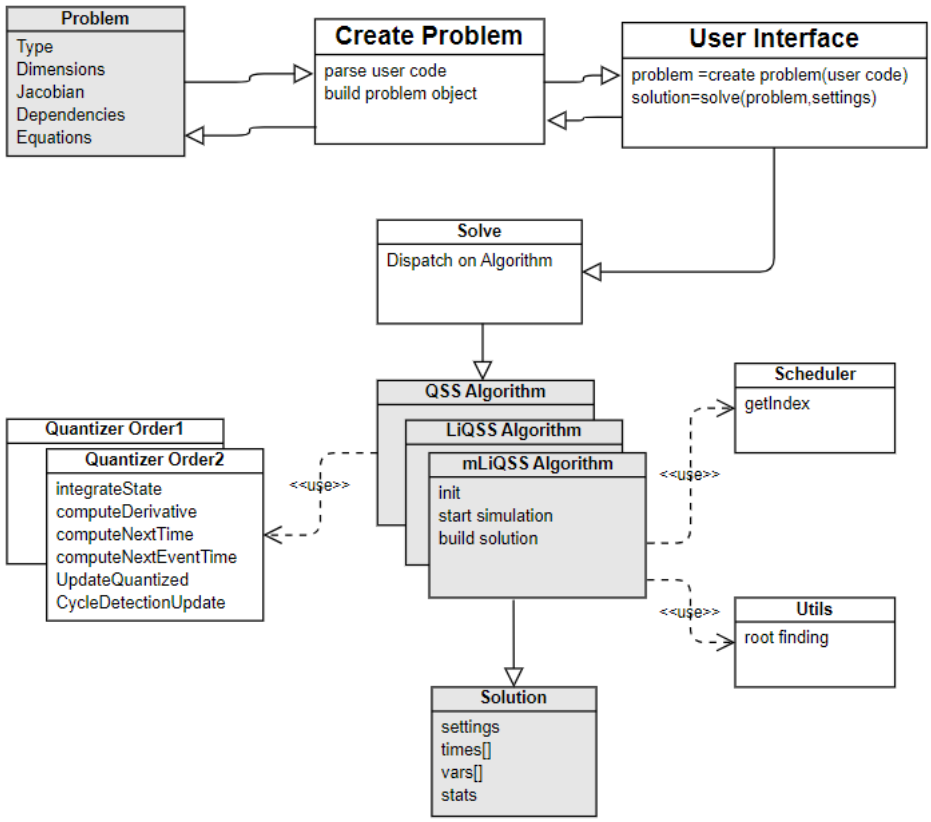

Figure 1: alt text

In addition, the package contains several shared helper functions used during the integration process by the algorithm such as the scheduler that organizes which variable of the system to update at any specific time of the simulation. The solver uses other packages such as `MacroTools.jl`(Innes, 2015) for user-code parsing, `SymEngine.jl`(Team, 2015) for Jacobian computation and dependencies extraction, and a modified `TaylorSeries.jl`(Benet & Sanders, 2014) that uses caching to obtain free Taylor variable operations as the current version of TaylorSeries creates a heap allocated object for every operation. The approximation through Taylor variables transforms any complicated equations to polynomials, which makes root finding cheaper, which the QSS methods relies heavily on it.

In conclusion, the package offers extensive functionality to facilitate practical research tasks, all while being lightweight and well documented to be easily used by researchers and students to efficiently model various dynamical systems with discontinuities, as well as further study and improve the newly developed QSS methods. In Fact, Anyone that has a different problem

type that can be handled by a specific QSS algorithm that outputs a different solution, they can define their types as subclasses of the abstract types (Problem, Algorithm, and Solution), and use the solver as usual.

Acknowledgements

This research has received no external funding.

References

Benet, L., & Sanders, D. P. (2014). *A Julia package for Taylor expansions in one or more independent variables*. https://github.com/JuliaDiff/TaylorSeries.jl. https://doi.org/10.5281/zenodo.2557003

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65–98. https://doi.org/10.1137/141000671

Innes, M. (2015). *MacroTools.jl*. https://github.com/FluxML/MacroTools.jl

Pietro, F., Migoni, G., & Kofman, E. (2019). Improving linearly implicit quantized state system methods. In *Simulation: Transactions of the Society for Modeling and Simulation International*.

Team, D. (2015). *SymEngine.jl*. https://github.com/symengine/SymEngine.jl

DRAFT